Successful node insertion: array, word and signed invariant contract

Producer T-1335; archive T-1337. The source is SignedGraph::node at fc45cda7bf. The return-state model appends the old length to the USize parent array, one to the size array, the identity potential to the potential array, and None to the anchor array. This packet connects those four modeled appends to the preceding abstract insertion construction.

Result. Ten named declarations compile. Old vector entries are retained; the new parent word is exactly n when $n <$ USize.size. All parent indices remain in bounds. Decoding the appended columns equals the abstract word insertion, and the appended size vector equals its prior model. Insertion preserves canonical potentials and anchors and every existing signed equation, for any assigned value of the new vertex.

Combined contract. NativeNode.insertion_contract preserves six properties together: canonical word potentials, canonical anchor pairs, signed graph and anchor equations, forest termination, component-owner correctness and exact stored root-size cardinalities. It assumes those properties of the input, a prime word modulus, a positive modulus, bounded existing parents, equal initial column lengths, and $n+1 <$ USize.size. The last bound retains the downstream union-size representability invariant. The new node is deliberately unanchored, so its value is unrestricted by insertion.

Checks. Six controls preserve an old anchor and parent, give the new node no anchor, set its size to one and potential to identity, and check an empty graph insertion has no anchor. Sixteen separate named axiom readbacks use only propext, Classical.choice and Quot.sound. Three malformed inventories are rejected. Lean rejects a deliberately false theorem claiming the new anchor is Some(0,0). All five failed development attempts remain recorded alongside the successful attempts and negative control.

Boundary. This is a hand-written Lean model of the successful return state, with source bytes bound to an immutable revision. It does not prove a Rust compiler refinement or allocation success, nor describe interruption between sequential pushes. Counter fields, which the source node body does not modify, are outside this Columns model. Independent source-fidelity review is pending. Imported compiled dependencies are reused and hash-bound; no independent binary/kernel recheck was performed. Pre/post hashes do not exclude transient concurrent edits.

Evidence and next construction. Module checks were bounded to one thread, 4096 MiB and 120 seconds each. The exact receipts and sources, including warnings and failures, are retained. No scientific campaign, timing measurement or budget renewal occurred. Next, compose the node contract with complete native relation dispatch, returned anchor constraints and counter accounting; separately obtain independent fidelity review of the published packets. The earlier reports remain immutable historical snapshots.